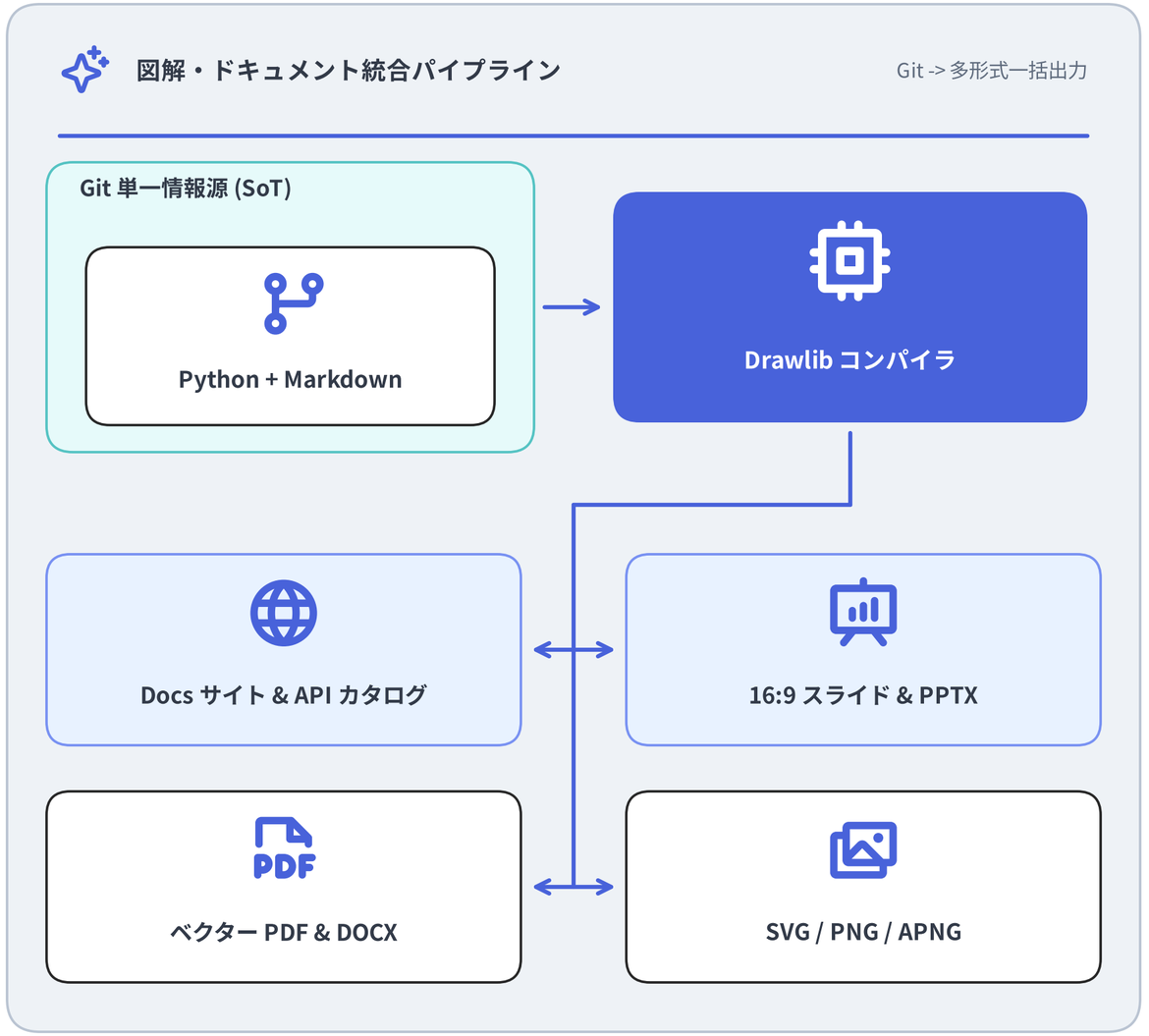
Drawlib

コードで描く図解と技術ドキュメント

ソフトウェアエンジニアとAIコーディングエージェントのために設計された、純粋Pythonの宣言的作図エンジン＆マルチターゲット文書コンパイラ。

公式ショーケースデッキ · v0.3.5 対応



数字で見る Drawlib の全体像

4,500+

ベクターアイコン

公式クラウド・汎用など全34スイート

38+

高水準ビジュアル

18 SmartArts・9 Charts
6 Diagrams・5 Graphs

5

プロジェクト型

site, catalog, doc, slide, images

100%

純粋Python & Git

GUIロックインなし・AI自己検証可能

Drawlib 4層アーキテクチャスタック

CLI基盤から多形式出力まで統合

L4

Layer 4・マルチターゲット・コンパイラ

HTML・Catalog・Slide・PDF・DOCX・PPTX

L3

Layer 3・宣言的ビジュアルエンジン群

SmartArts・Charts・Diagrams・Graphs・Anim

L2

Layer 2・基本図形・34種アイコン群・スタイル

43種の幾何図形・4,500+ ベクターアイコン・意味論テーマ

L1

Layer 1・AI エージェント CLI 基盤

rules 参照・show -g グリッド検証・SQLite キャッシュ

本プレゼンテーションの構成

全6章のロードマップ

- **背景と課題 (第1章)**: 図解の形骸化を防ぐ Python × Git の統合アプローチ
- **描画エンジン (第2〜4章)**: 基本図形・34種アイコン群・SmartArts・Charts・構成図
- **多形式出力とAI連携 (第5〜6章)**: 5つのプロジェクト型・Office出力・AI自己検証ループ



CHAPTER

01

Chapter 1 / 6

なぜ Drawlib なのか？

AI時代の技術図解・ドキュメント分断を解決する

1.1 従来ツール（GUI・DSL）の限界

1.2 コードによる図解と文書の統合

1.3 既存ツールとの機能比較マトリクス

1.4 単一リポジトリからの多形式一括出力

従来の作図・ドキュメントツールの限界

開発現場は **差分管理できないGUIツール、表現力が限定されたテキストDSL、文書と分断された描画ライブラリ** の三択を強いられ、リリースのたびに設計図が実装から乖離します。

GUI ツール (Figma / Draw.io)

- **不透明なバイナリ形式**
  Git PR 上で差分比較が不可能
- **手動の PNG 書き出し**
  仕様変更ですぐ陳腐化する
- **GUI 操作への依存**
  AI エージェントが編集不可

{ } テキスト DSL (Mermaid 等)

- **硬直的な自動配置**
  (x, y) 座標の微調整が不可
- **狭い図解ボキャブラリ**
  KPI・表・地図表現に不向き
- **図単体のスニペット**
  PDF / PPTX 統合出力がない

描画ライブラリ (Matplotlib)

- **数値プロット専用**
  クラウド構成図や UML 非対応
- **煩雑なスタイル記述**
  書体や配色の統一が困難
- **ドキュメントとの分断**
  資料へ手作業で画像を転記

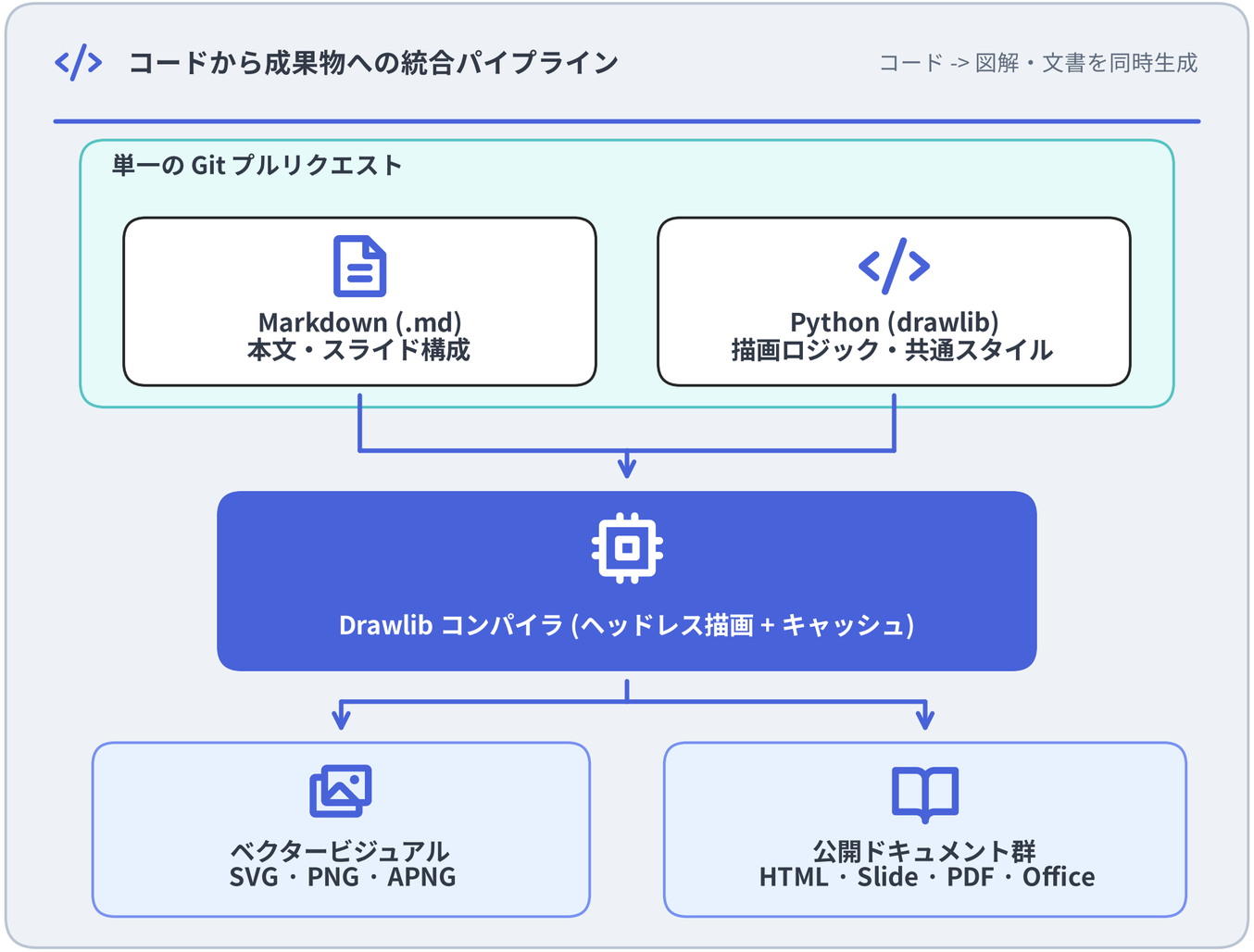
求められていた統合基盤：精密な空間制御 × Git差分レビュー × 文書・図表統合 × AI視覚検証ループ

Illustration & Documentation as Code

Python と Markdown の統合

- **チューリング完全な記述**: ループ・変数・`utils.py` 共通関数を活用
- **Markdown 直書き**: 原稿内の `drawlib` ブロックから直接描画
- **決定論的な PR 差分**: 文章と図解の更新を同一コミットでレビュー

```
setup(width=120, height=32)
p = ChevronProcess(style=Styles.Neutral)
for s in ("記述", "変換", "公開"): p.add(s)
p.draw((6, 8), width=108, height=16)
save()
```

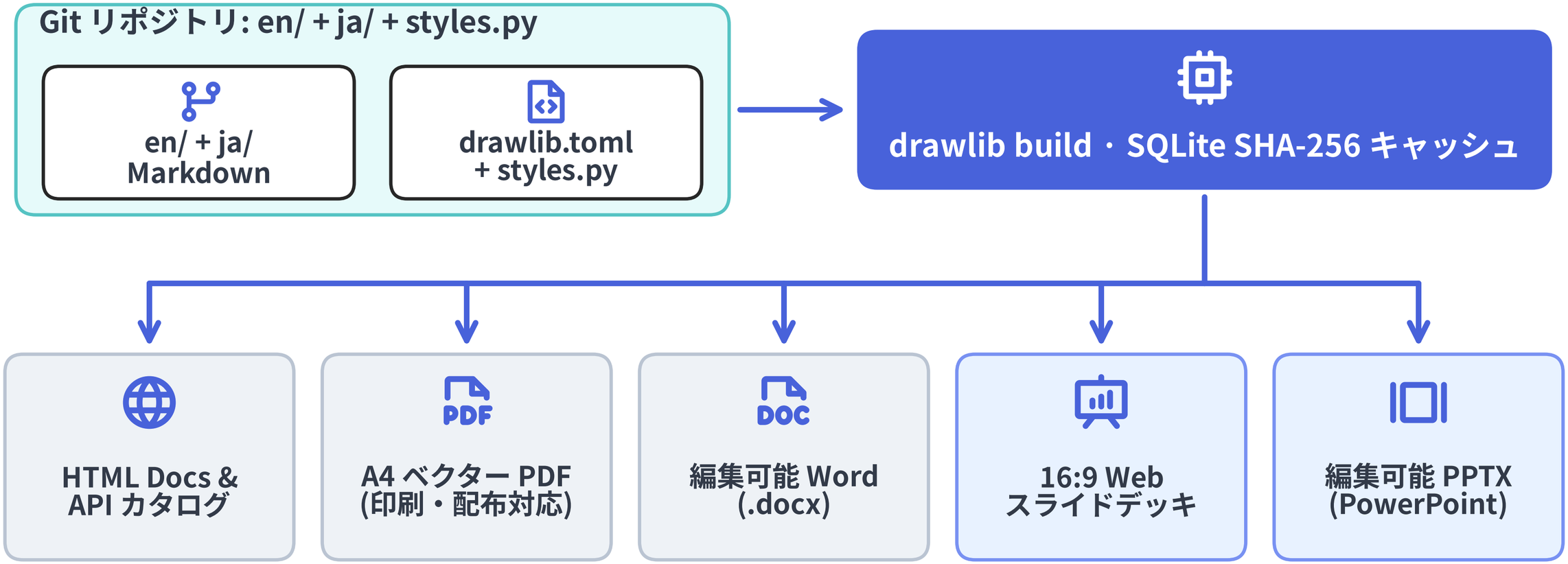


既存ツールとの機能比較マトリクス

評価軸	GUI ツール	テキスト DSL	グラフ描画系	Drawlib
Git 差分・PR レビュー	× バイナリ不可	✓ テキスト管理	✓ Python コード	✓ 明快な Python
空間座標の精密制御	✓ 手動ドラッグ	× 自動配置のみ	△ グラフ軸のみ	✓ 座標指定＋自動
クラウド構成図・UML	✓ 手動アイコン	△ 簡易ボックス	× 非対応	✓ 6種エンジン内蔵
チャート・情報図解	△ 手動で作図	× ごく一部のみ	✓ 統計図のみ	✓ Charts & SmartArts
Site / Catalog / Slide / Office	× 画像書き出し	× 図単体のみ	× 画像出力のみ	✓ 5型マルチ出力
AI 視覚自己検証 (-g)	× GUI 操作必須	× 座標調整不可	△ 個別実装のみ	✓ 標準 -g CLI

単一リポジトリからあらゆる形式へ一括出力

単一の `drawlib.toml` と `[i18n]` 構造パリティ検証により、1つのGitコミットから日英同期された **Web・印刷・Office** の全成果物を一括コンパイルします。



CHAPTER

02

Chapter 2 / 6

基本図形・スタイル・アイコン

43種の図形・8色セマンティックテーマ・4,500超のアイコン

2.1 直交座標キャンバスと -g 検証グリッド

2.2 43種のプリミティブ図形とコネクタ

2.3 多言語タイポグラフィとフォント体系

2.4 34アイコンスイートと8色テーマ

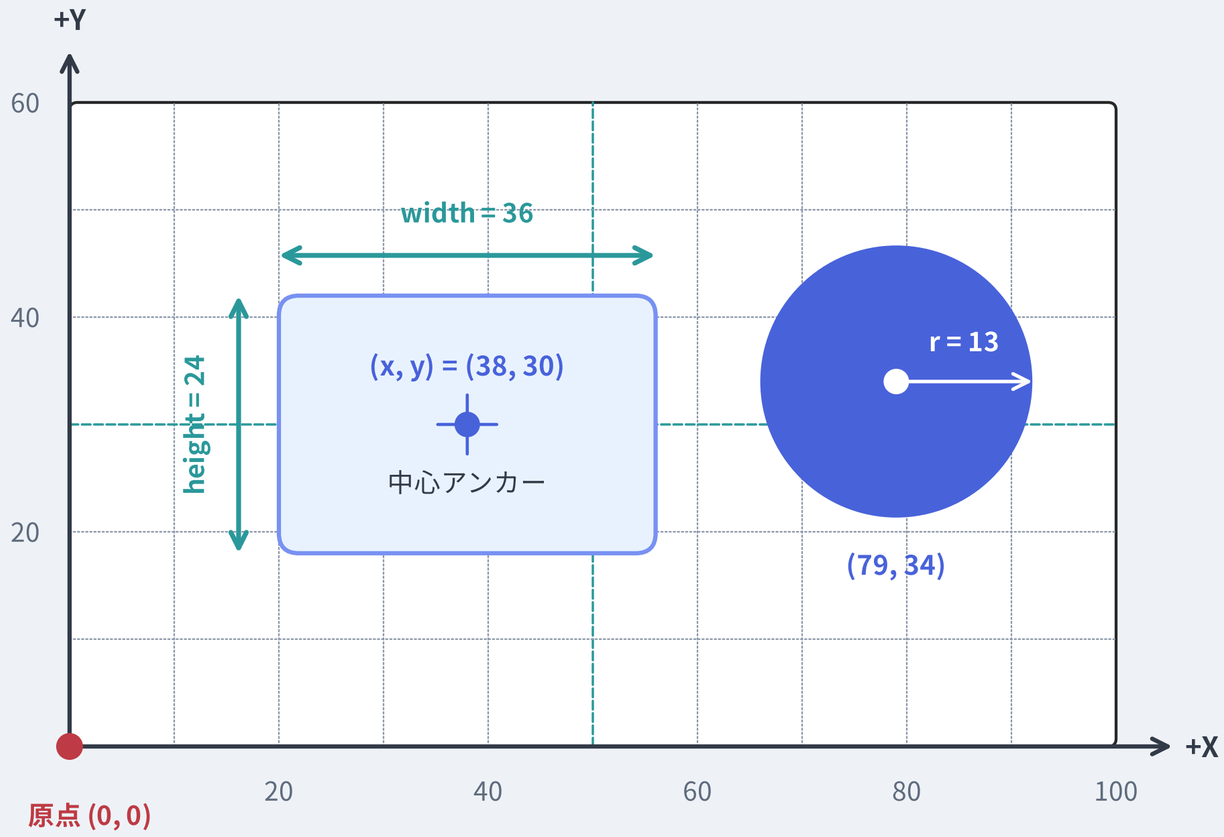
直交座標キャンバスと検証グリッド (-g)

直感的で予測可能な座標設計

- **左下原点** `(0, 0)`: 数学と同じ右が `+X`・上が `+Y` の直交座標系。
- **中心アンカー統一**: 図形・文字・アイコンすべて `(x, y)` が中心点。
- **検証グリッド (`-g`)**: 目盛り線オーバーレイで配置ズレを即座に確認。

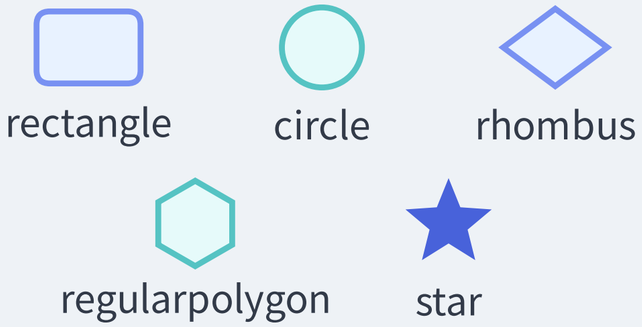
```
setup(width=100, height=60)
st = Styles.PrimaryFlat
rectangle((50, 30), 36, 22, style=st)
save()
```

直交座標キャンバス: setup(width=100, height=60) + グリッド (-g)



43種のプリミティブ図形とコネクタ

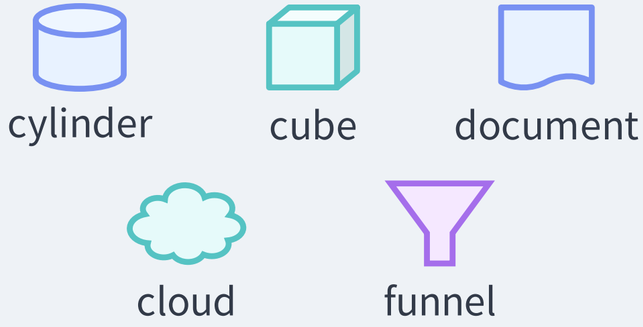
1. 基本図形・多角形



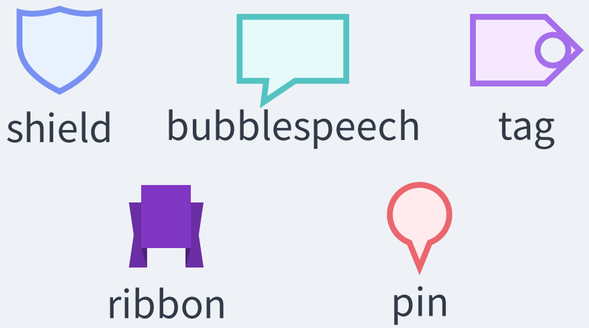
2. ブロック矢印・フロー



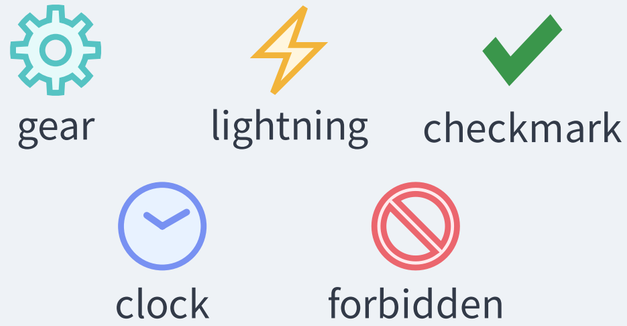
3. 3D・コンテナ図形



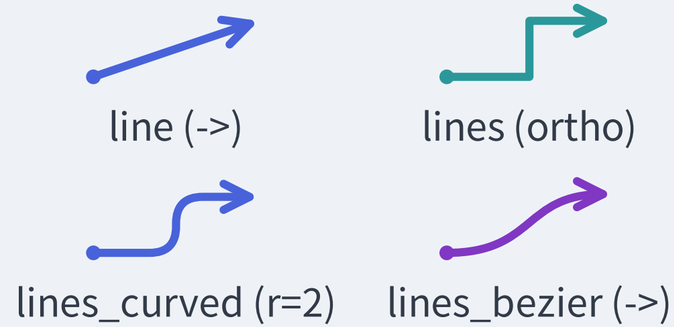
4. バッジ・吹き出し



5. ドメイン記号



6. 線・折れ線・曲線コネクタ



多言語タイポグラフィと画像処理

多言語フォントと Dimage 処理

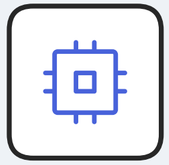
- **和欧混植に標準対応**: 標準 `Font` で英数字と日本語を同時に描画。
- **地域別・等幅パック**: `FontJapanese`・`FontArabic`・`FontSourceCode`。
- **精密アンカー制御**: `halign` / `valign` で基準点からの配置を指定。
- `Dimage` **画像処理**: 切り抜き・セピア・モザイクと自動キャッシュ。





34スイート・4,500種超のベクターアイコン

1. Phosphor ベクターアイコン (5ウェイト)

thin	light	regular	bold	fill
				
rocket_launch	cpu	globe	sparkle	shield_check

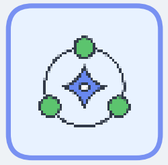
2. 公式クラウドスイート (gcp・aws・azure)

gcp	gcp	aws	aws	azure
				
Cloud Run	BigQuery	Lambda	Amazon S3	Cosmos DB

3. 15種の公式ドメインブランドスイート

ai	cnative	devops	data	security
				
Gemini	Kubernetes	GitHub	PostgreSQL	Cloudflare

4. 15種のベンダー中立2Dスイート (original_*)

cloud	ai	server	network	security
				
multi_cloud	ai_agent	gpu_server	firewall	zero_trust

8色セマンティックスタイルとプリセットテーマ

- **8色ロール体系**: `Styles.<Color><Variant>` が `Primary` 〜 `Dark` の8色と `Flat` ・ `Bold` ・ `Dashed` ・ `Neutral` を統合。
- **不変 `.patch()` とテーマ**: `style.patch(...)` で安全に派生作成し、 `drawlib.toml` で全体の配色テーマを一括切替。

8色ロール × バリアント行列 (Styles.<Color><Variant>)

ロール	Default	Flat	Neutral / Dashed
Primary	.Primary	.PrimaryFlat	.PrimaryNeutral
Secondary	.Secondary	.SecondaryFlat	.SecondaryNeutral
Accent	.Accent	.AccentFlat	.AccentNeutral
Success	.Success	.SuccessFlat	.SuccessNeutral
Warning	.Warning	.WarningFlat	.WarningNeutral
Danger	.Danger	.DangerFlat	.DangerNeutral
Neutral	.Neutral	.NeutralFlat	.NeutralDashed
Dark	.Dark	.DarkFlat	.DarkDashed

プリセットテーマ (drawlib.toml)

style = "default"
標準インディゴ配色



style = "google"
Google ブルー配色



style = "monochrome"
印刷向けグレースケール



CHAPTER

03

Chapter 3 / 6

SmartArts と標準チャート

18種の構造化インフォグラフィックと9種の純Pythonチャート

3.1 プロセス・サイクル・ロードマップ

3.2 KPI・カードリスト・階層・マインドマップ

3.3 テーブル・コード・カレンダー・世界地図

3.4 9種の宣言的チャートとガント工程表

プロセス・サイクル・ロードマップ

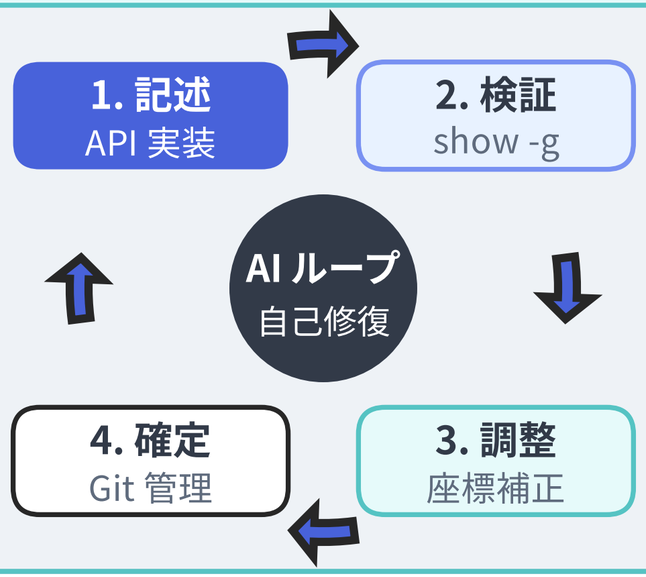
ChevronProcess — 段階的ビルド・配信パイプライン

4ステージ工程

1. **Author**
   MD + Python 記述
2. **Render -g**
   グリッド視覚検証
3. **Cache**
   SQLite 差分キャッシュ
4. **Publish**
   HTML / PDF / PPTX

Cycle — 自律改善ループ

4ノード円軌道



Roadmap — マイルストーン工程

曲線スパイン

- **P1** ベクター描画基盤
  図形プリミティブ・アイコン・テーマ
- **P2** SmartArts・標準チャート
  18種の構造化図解と9種のグラフ
- **P3** ダイアグラム・自動配置
  6種の図解エンジンとレイアウト計算
- **P4** マルチ形式ビルドとCLI
  HTML・PDF・PPTX出力とAIルール

KPIカード・リスト・階層・マインドマップ

KpiCards — 指標グリッド

2x2 マルチ列

18	**SmartArt 種別** 表から世界地図まで
34	**アイコンスイート** 主要クラウド網羅
0	**GUI 依存ゼロ** 100% コード管理
4	**標準テーマ数** 一括パレット切替

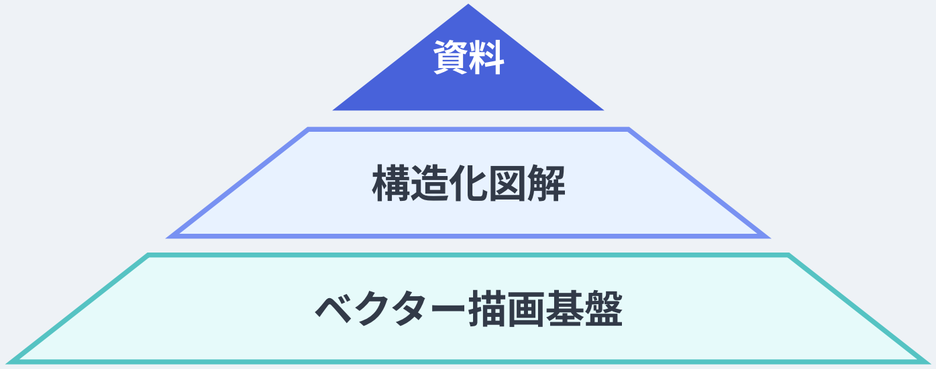
CardList — 機能カードリスト

アイコン + バッジ

- L1 **宣言的な要素ライフサイクル**
  .add() で登録し .draw() で一括描画
- L2 **セマンティックな配色トークン**
  落ち着いたベース色と焦点ハイライト
- L3 **アニメーション対応の表示制御**
  座標を崩さず item.show で段階表示

Pyramid — アーキテクチャ階層

3段階レイヤー



MindMapNode — 放射ツリー

2パス自動配線



テーブル・コード・カレンダー・世界地図

Table — ベンチマーク比較表

ゼブラ配色

出力ターゲット	フォーマット	初回ビルド
16:9 スライド資料	HTML / PDF	0.42 秒
技術ドキュメント	複数ページ	0.65 秒
単体ベクター図解	SVG / PNG	0.18 秒

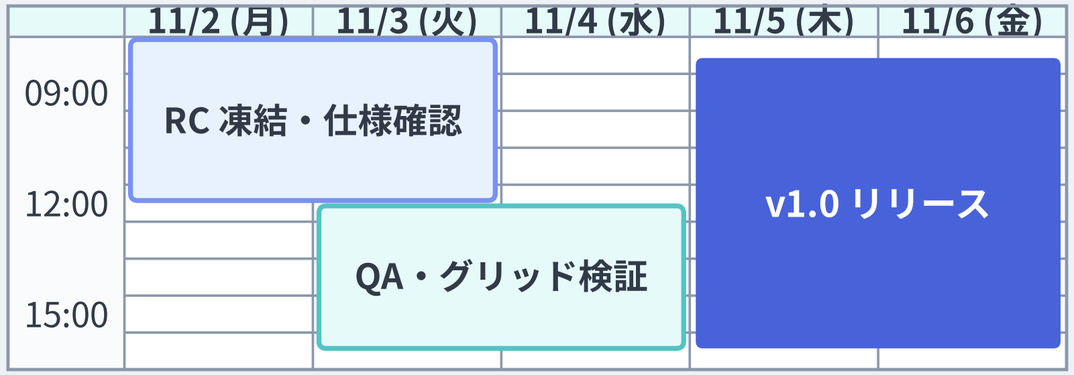
SourceCode — 構文強調

card_demo.py

```python
from drawlib.canvas import save, setup
from drawlib.shapes import rectangle
from drawlib.styles import Styles
setup(width=120, height=60)
rectangle((60, 30), 80, 36, Styles.Primary)
save("card.svg")
```

WeekCalendar — スプリント計画

ISO 8601



GeoMap — 世界地図

US / EU / JP



宣言的な棒・折れ線・面チャート

9

標準内蔵チャート

棒・折れ線・面・円・レーダー
散布図・ヒートマップ・ガント

100%

テーマ完全同期

Default / Google / Monochrome
各パレットへ自動追従

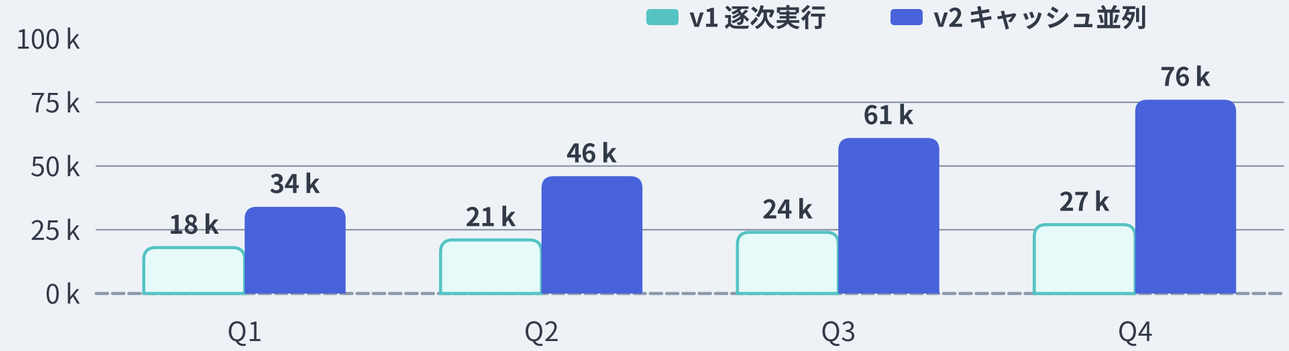
Auto

目盛の自動最適化

見やすい軸目盛・独立凡例と
draw_ratio アニメーション

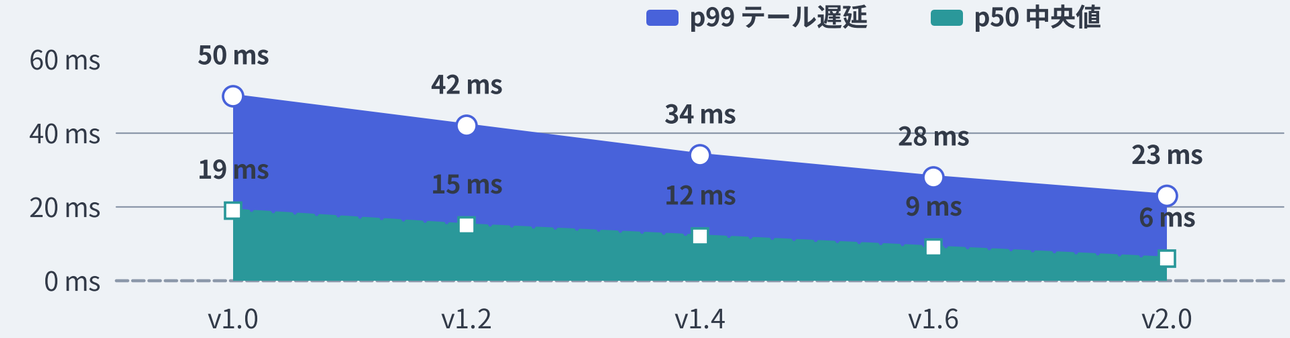
BarChart — 四半期別レンダリング処理量 (k/時)

グループ棒



AreaChart — ビルド遅延の推移 (p50 vs p99)

平滑スプライン



円・レーダー・散布図・ヒートマップ

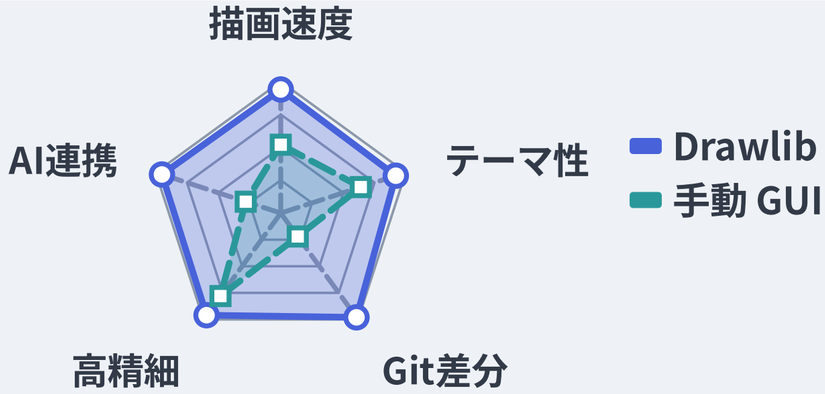
PieChart — ドーナツ構成比

hole_ratio=0.55



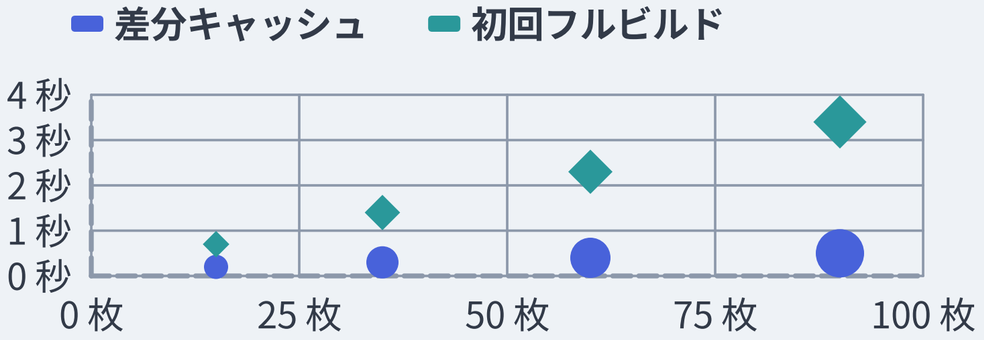
RadarChart — 5軸特性評価

ポリゴン網



ScatterChart — ビルド時間相関

バブル系列



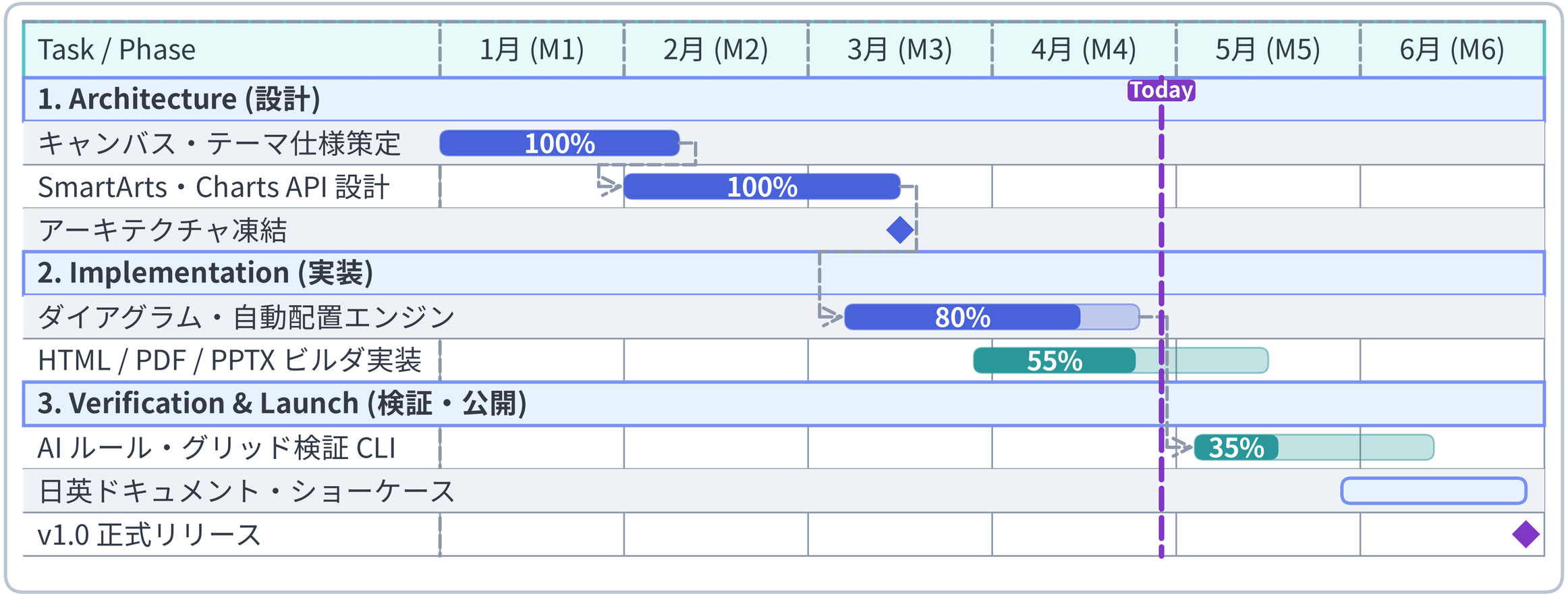
HeatmapChart — 遅延分布

輝度反転

	00:00	06:00	12:00	18:00	22:00
米国西部	14ms	18ms	42ms	35ms	19ms
欧州西部	16ms	29ms	54ms	38ms	21ms
東京拠点	22ms	46ms	68ms	49ms	25ms
豪州東部	18ms	34ms	51ms	41ms	22ms

GanttChart によるプロジェクト工程管理

- **宣言的なスケジュール定義**: `Section` による工程分割、`Task(progress=...)` の進捗バー、`Milestone` をコードで一元管理。
- **依存関係と基準日マーカー**: タスク間の直交 `Dependency` 矢印や、`Today`・`Release` などの垂直 `Marker` を自動描画。



CHAPTER

04

Chapter 4 / 6

技術ダイアグラム・自動配置・アニメ

6種の設計図エンジン・5種のグラフソルバー・APNG

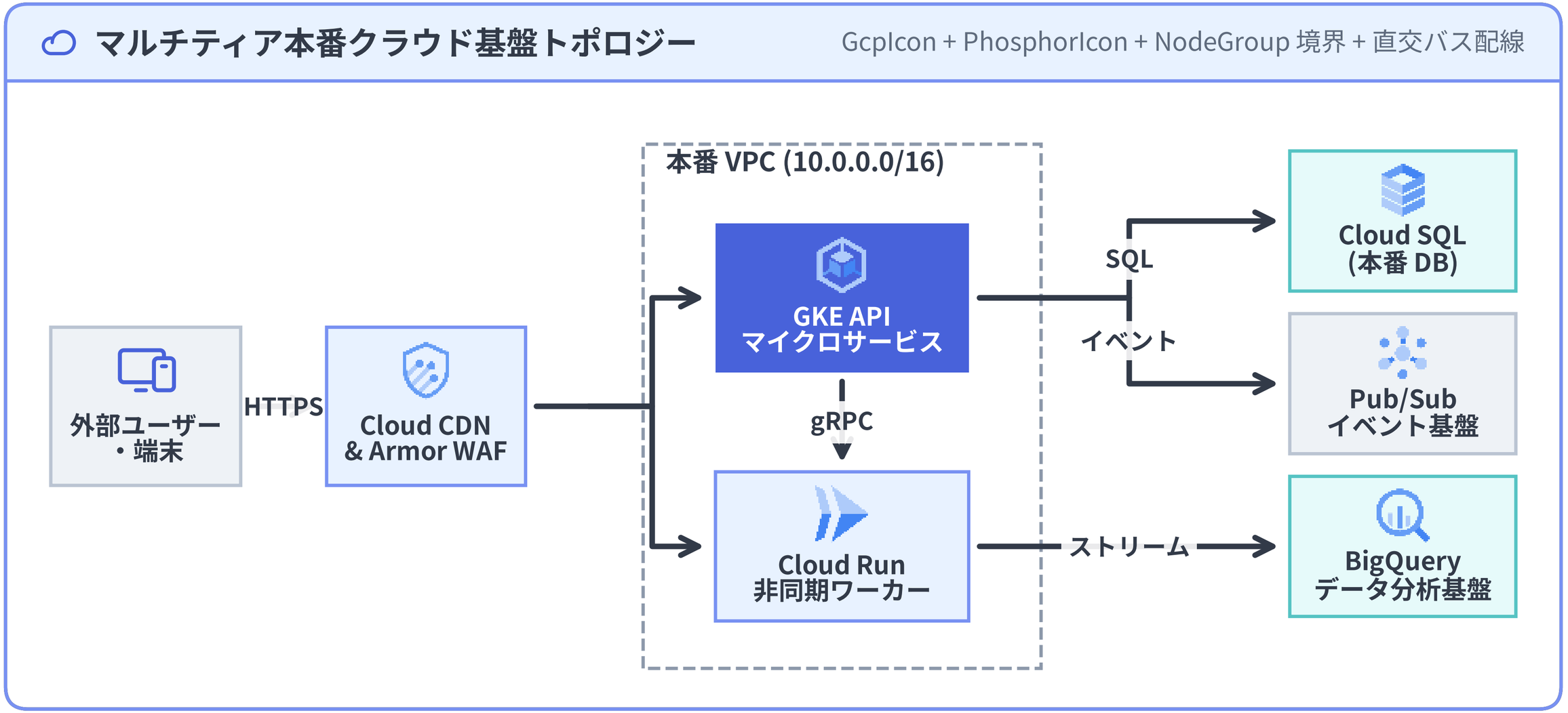
4.1 クラウド構成・ネットワーク図

4.2 業務フロー図と通信シーケンス図

4.3 UMLクラス図・ER図・状態遷移図

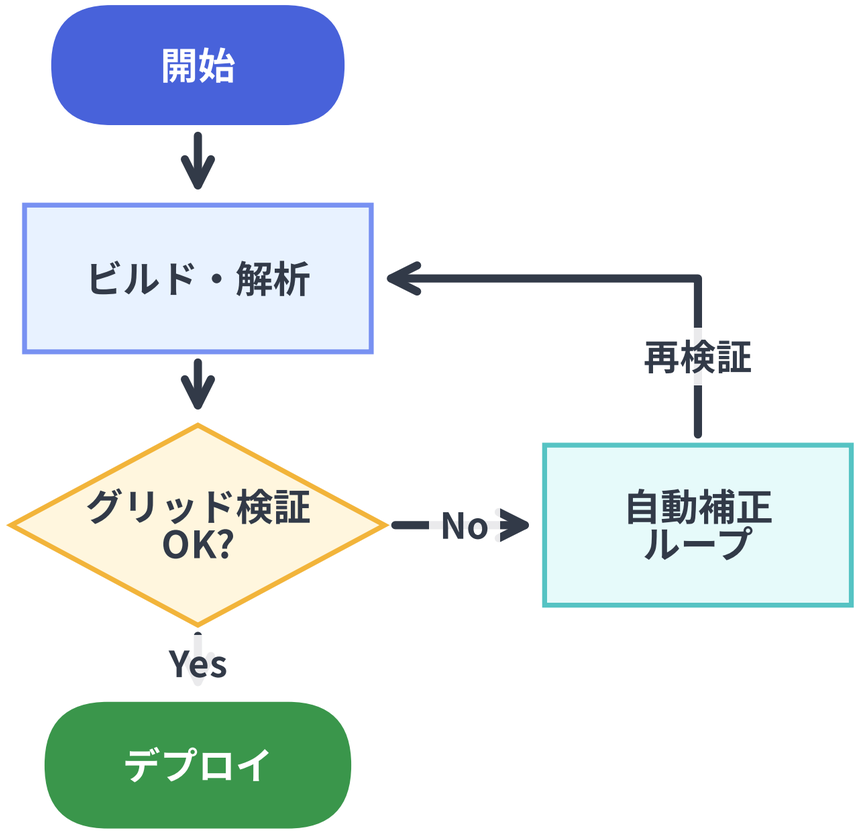
4.4 自動グラフレイアウトとアニメーション

ArchitectureDiagram によるクラウド構成図

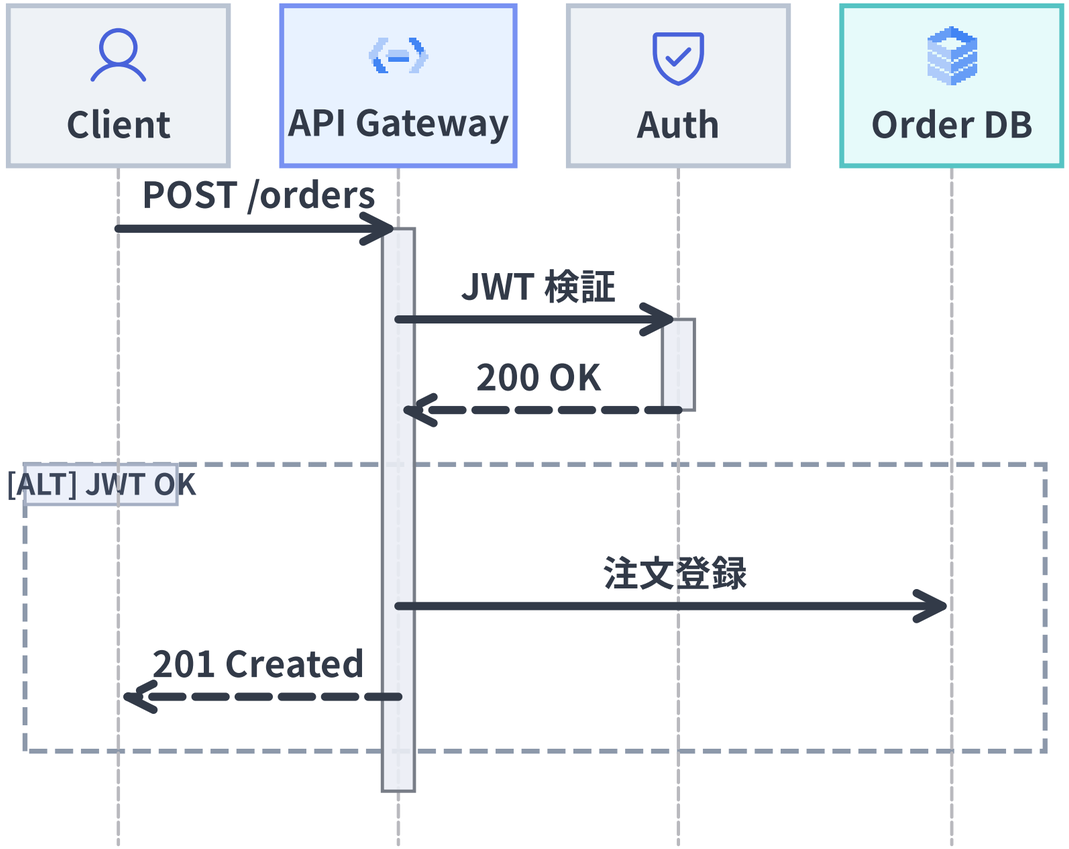


業務フローと通信シーケンス図

FlowDiagram — CI/CD 検証フロー

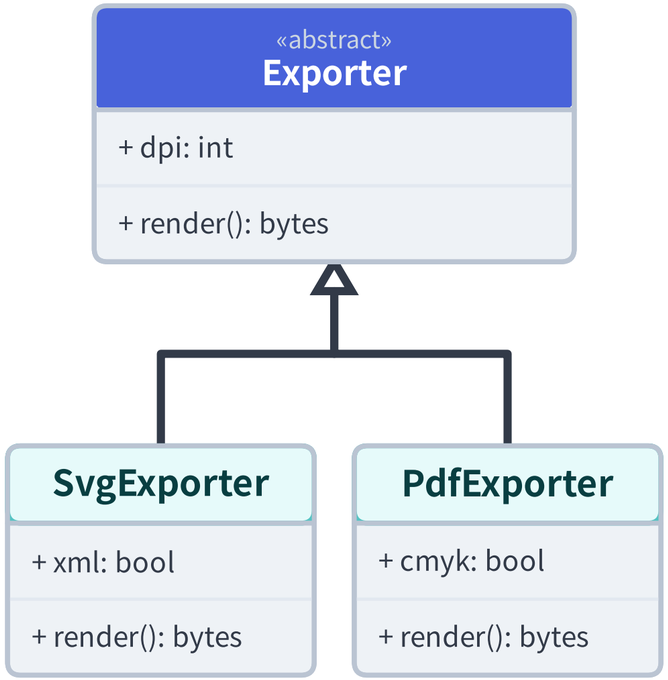


SequenceDiagram — API 通信順序



ドメイン設計：クラス図・ER図・状態遷移図

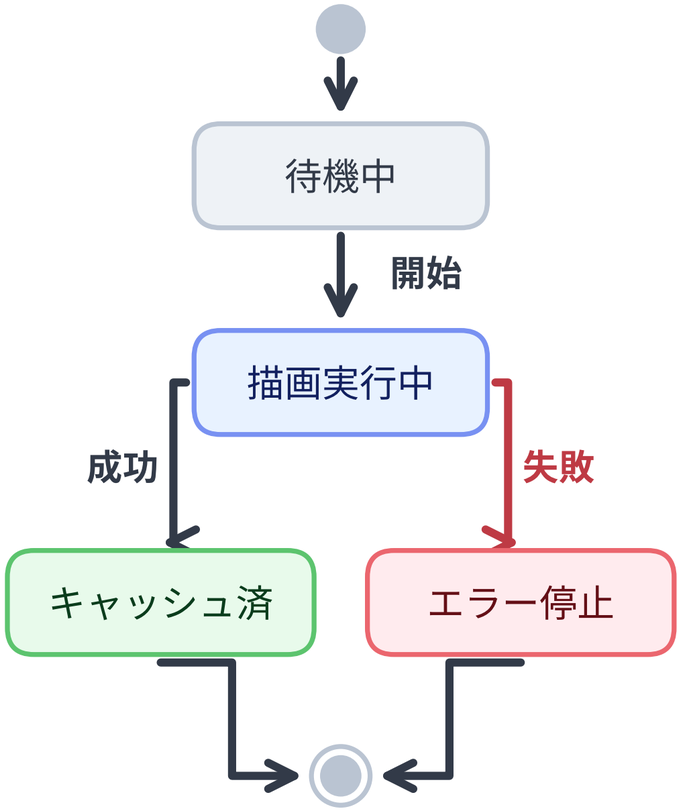
ClassDiagram (クラス図)



ERDiagram (ER図)



StateDiagram (状態遷移図)

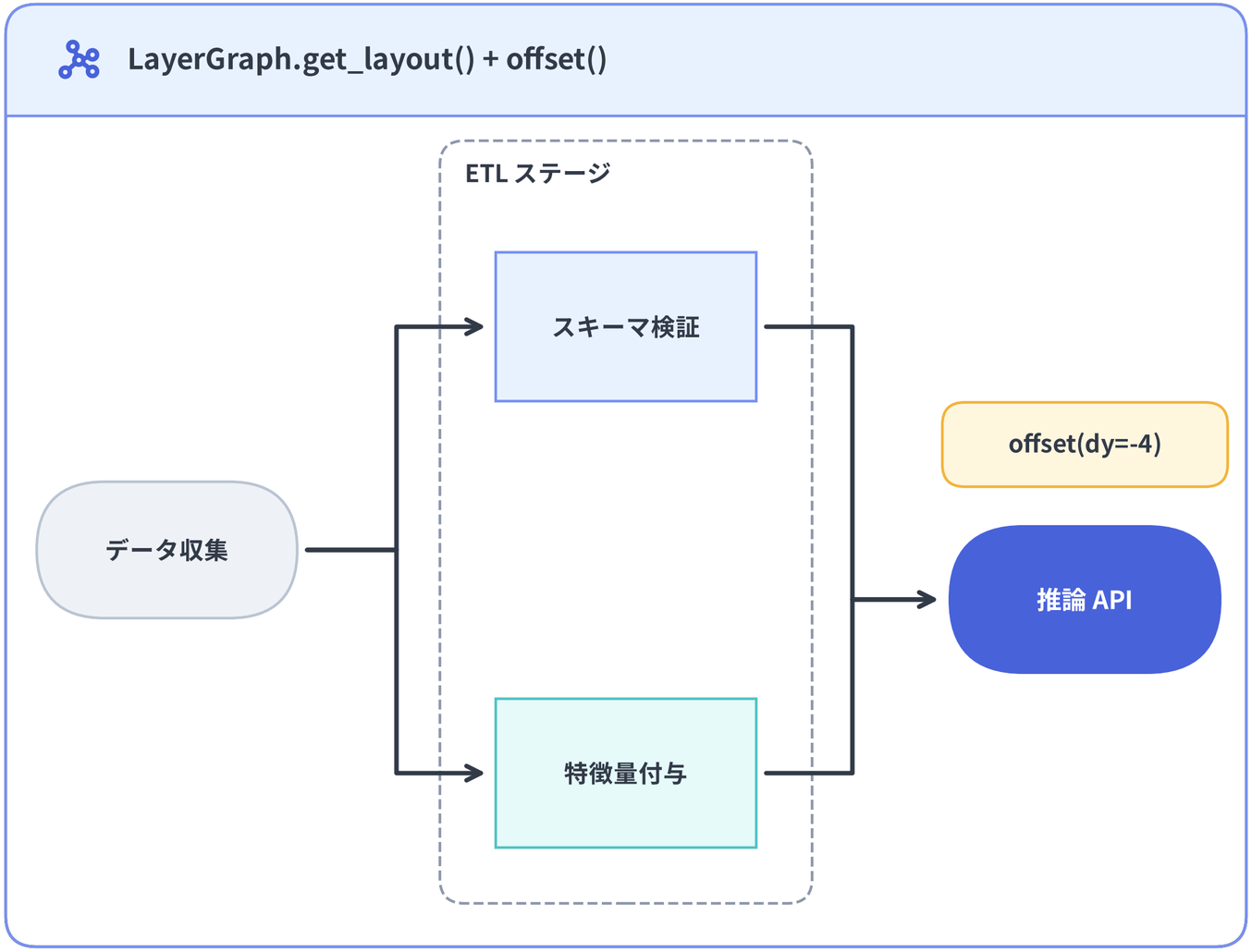


自動座標ソルバー (drawlib.graph)

自動配置と手動微調整のハイブリッド

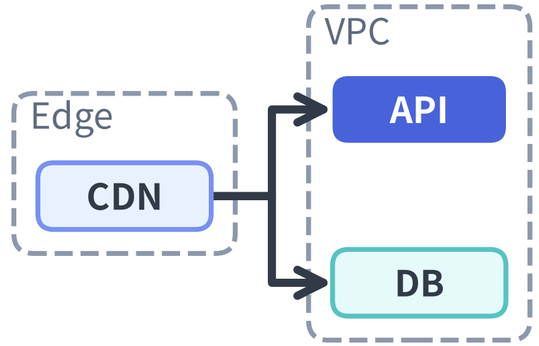
- **自動座標計算**: `get_layout()` で交差のない `(x, y)` を算出
- **手動微調整**: `offset(dx, dy)` でブラックボックス化を回避
- **確定座標の出力**: `print(layout)` で座標を確認しコード固定

```
g = LayerGraph(direction="LR")
g.edge("ingest", "validate")
g.edge("validate", "serve")
layout = g.get_layout(margin=8.0)
layout.offset("serve", dy=-4.0)
```

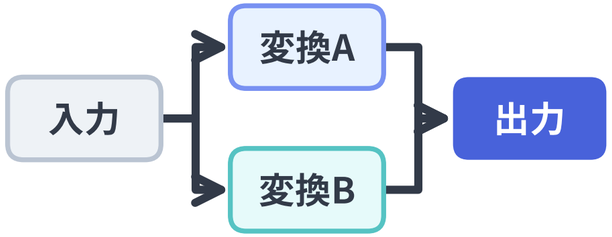


5種類の専用グラフレイアウトソルバー

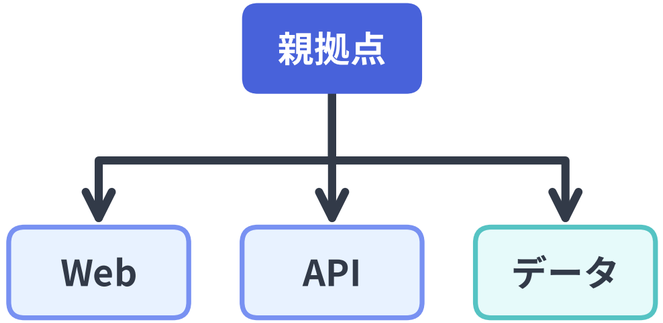
1. ArchitectureGraph (境界)



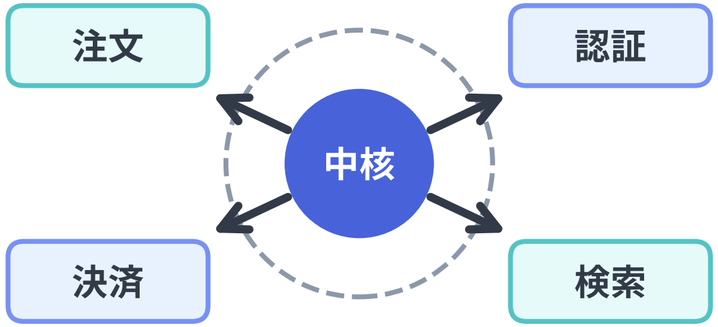
2. LayerGraph (階層型 DAG)



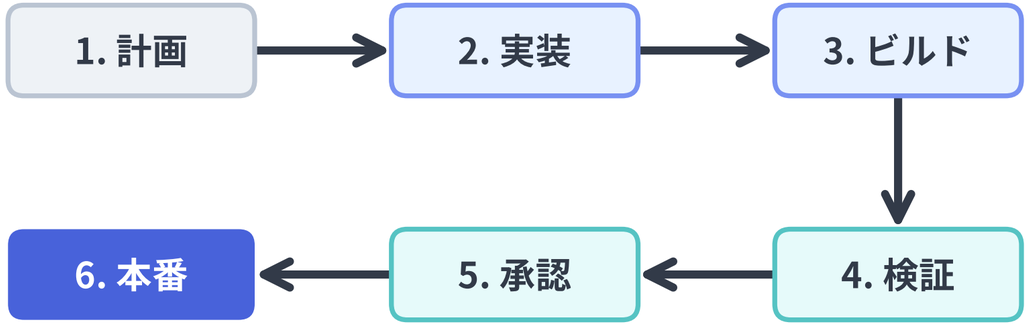
3. TreeGraph (整列ツリー)



4. RadialGraph (同心円ハブ＆スポーク)



5. GridGraph (2D マトリクス・蛇行フロー)

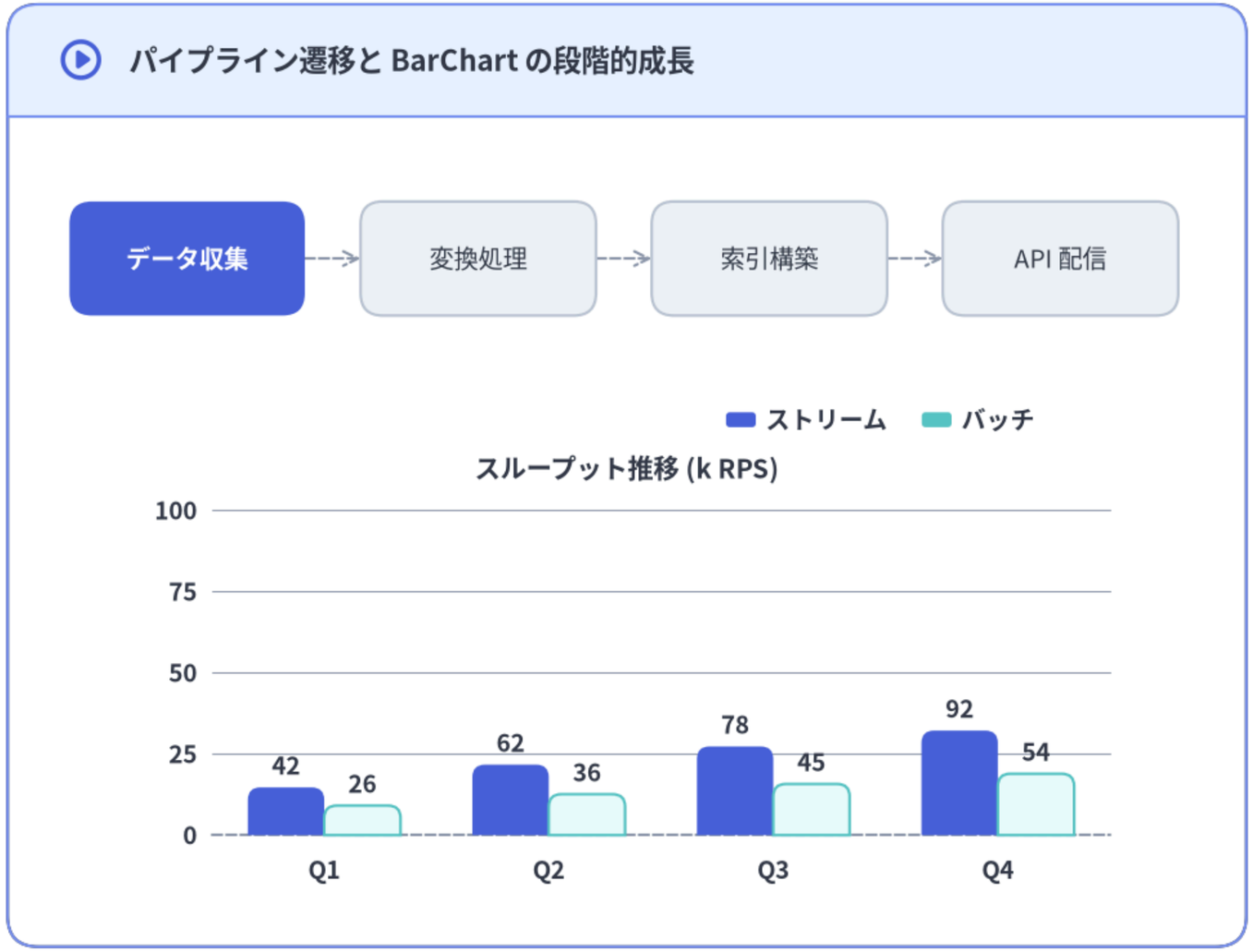


フレームベースアニメーション (APNG / WebP)

全コンポーネント対応の宣言的アニメーション

- **マルチフレーム制御**: 宣言的な `Animation(fps, loop)` エンジン
- **段階的描画 (`draw_ratio`)**: 線・SmartArt・チャート・設計図に対応
- **対話型再生制御**: `<canvas>` 上で `anim_trigger`・`anim_pause`・`A` 操作
- **静的出力との両立**: PDF・PPTX 変換時に最適フレームを自動抽出

```
anim = Animation(fps=2.0, loop=0)
for r in [0.4, 0.7, 1.0]:
    series.draw_ratio = r
    with anim.frame():
        chart.draw(xy=(10, 6))
```



CHAPTER

05

Chapter 5 / 6

5つのプロジェクト種別と出力

Site・Catalog・Doc・Slide・Images を単一コンパイラで

5.1 5つのプロジェクト種別の全体像

5.2 Web公開：SiteとCatalog

5.3 技術白書：HTML・PDF・Word出力

5.4 16:9スライド・発表者画面・PPTX

用途に応じた5つのプロジェクト種別

`uv run drawlib init project <type> <dir>` 単一コンパイラ • 5つの専用テンプレート • Web • PDF • Office対応

site

`docs_src/<lang>/`

体系的ガイドサイト
navbar.md・全文検索
ダークモード・多言語

例: www.drawlib.com

HTML | Search | SVG

catalog

v0.3.5 新機能

`catalog_src/<lang>/`

2カラムAPI辞書
H1/H2自動カテゴリ
/ キー即時フィルタ

例: drawlib.com/api

HTML | / Filter | SVG

doc

`doc_src/<lang>/`

単一長文仕様書・白書
--generate-index
表紙・目次自動生成

設計書・RFC・提案書

HTML | PDF | DOCX

slide

`slide_src/<lang>/`

16:9 スライド画面
?presenter=1 同期
::: note 発表原稿

登壇資料・勉強会

HTML | PDF | PPTX

images

`images_src/*.py`

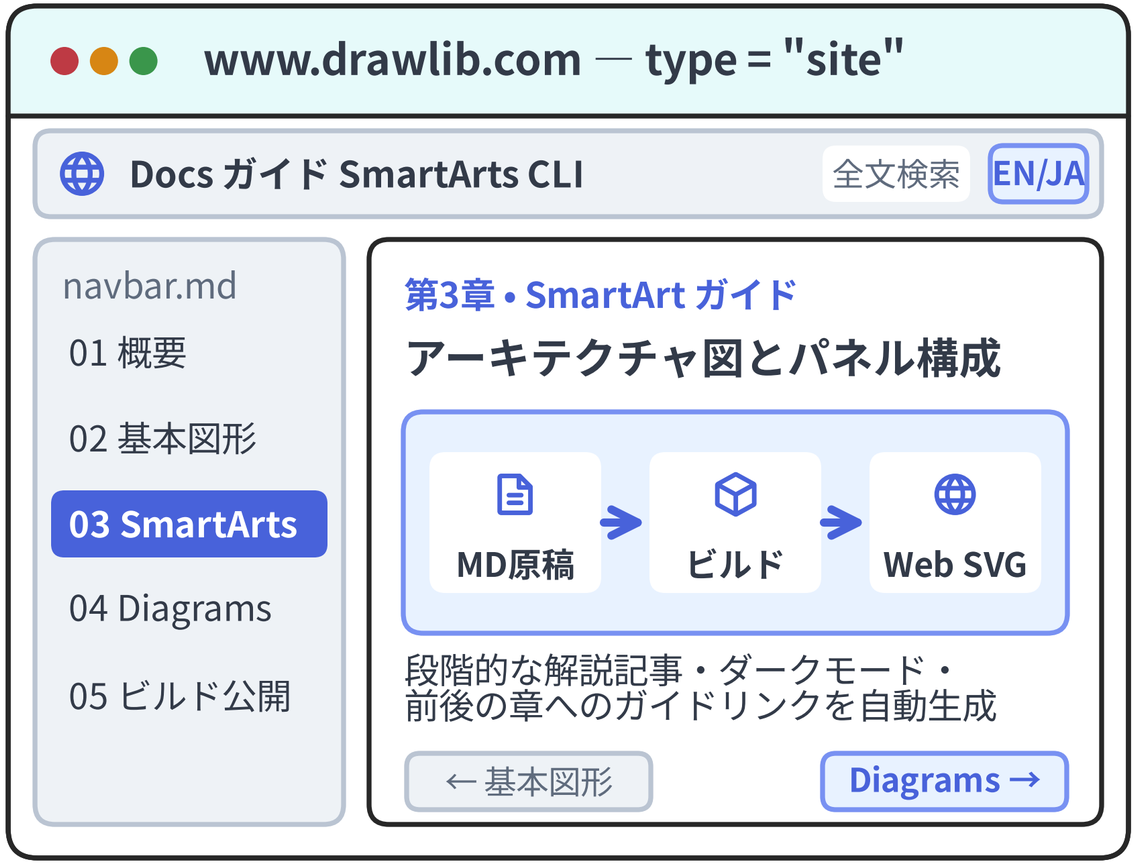
Python図解バッチ
ベクター & ラスター
CIアセット一括生成

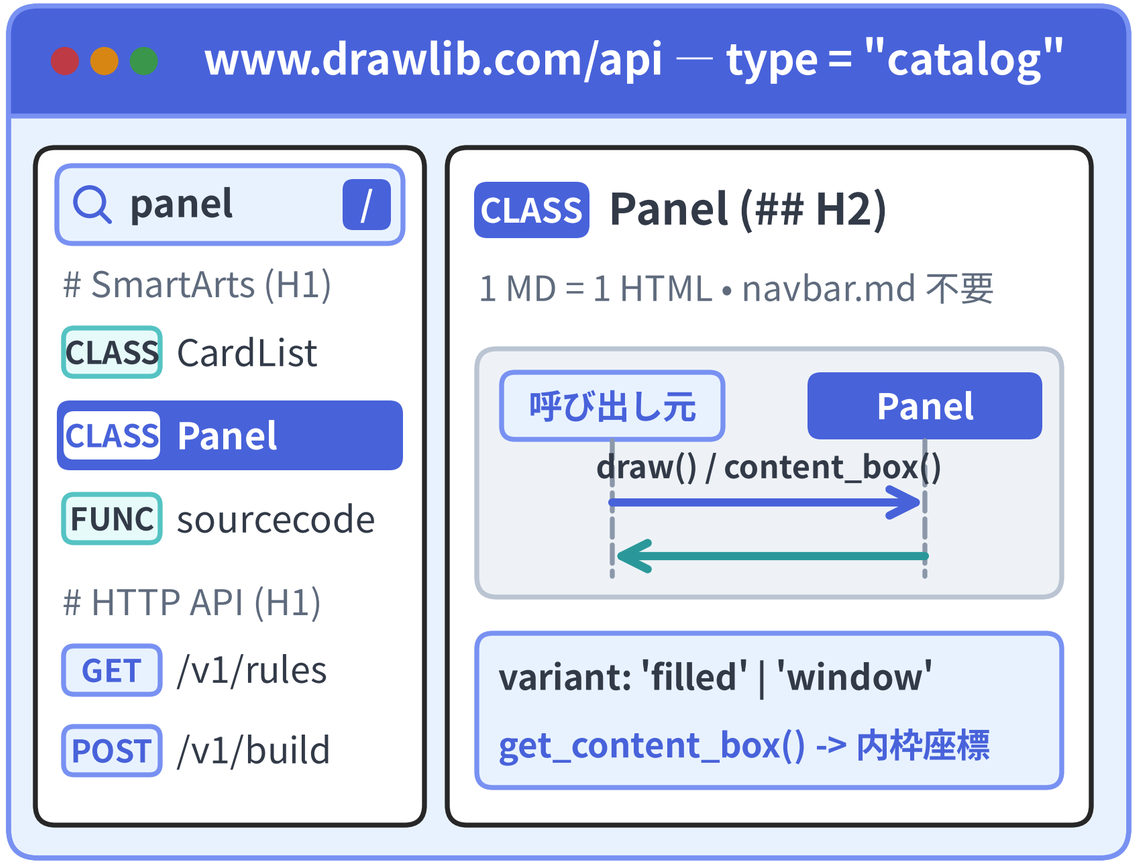
README・論文図版

SVG | PNG | WebP

Web公開：ドキュメント Site と API Catalog

- `type = "site"` **（体系的ドキュメント）**：`navbar.md` による章構成・全文検索・ダークモード・多言語切替を備えたガイドサイト（`www.drawlib.com`）。
- `type = "catalog"` **（API・スキーマ辞書）**：`navbar.md` 不要で `# H1` / バッジ付き `## H2` を自動抽出し `/` で即時絞り込む2カラム辞書



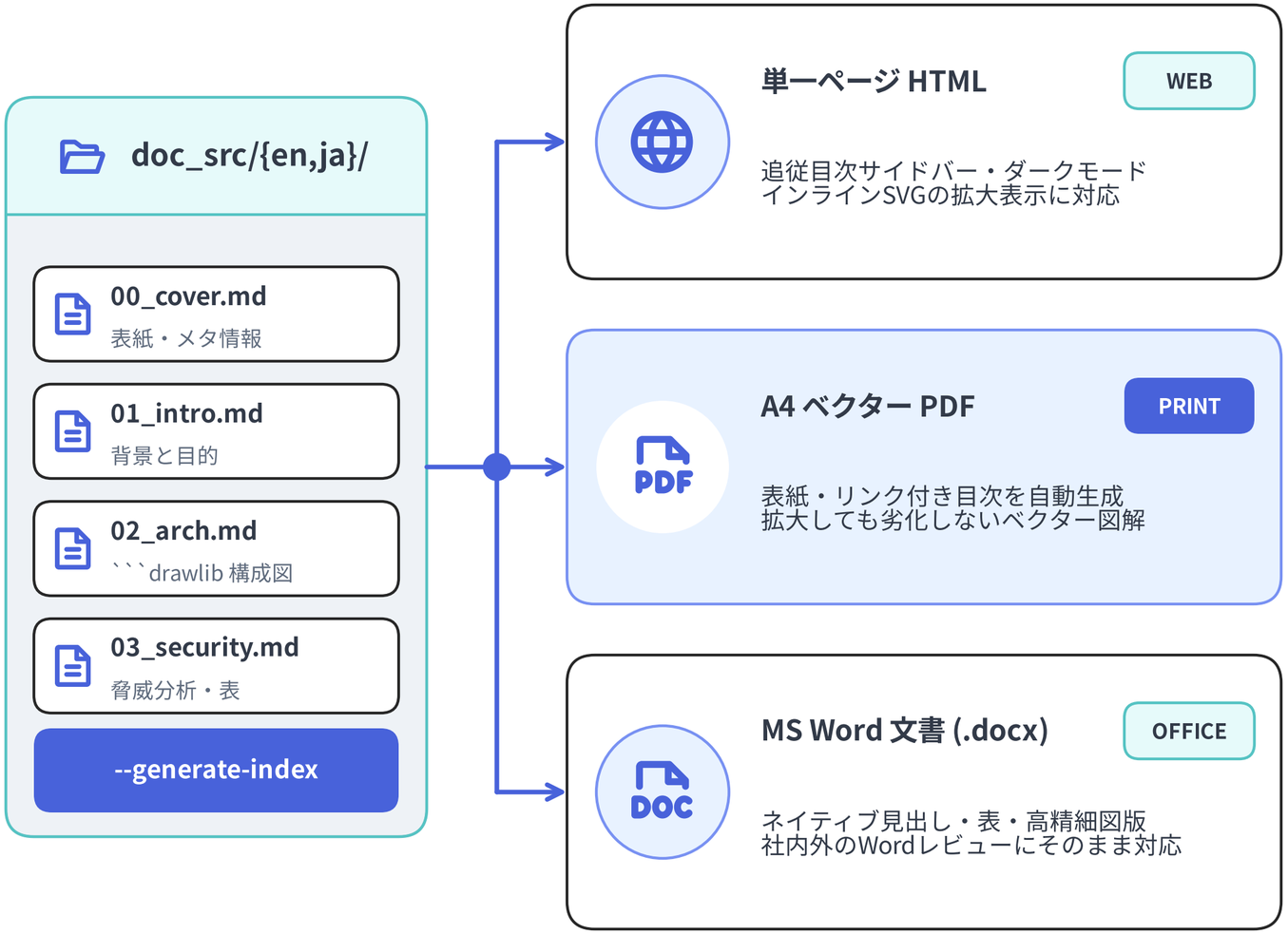


技術白書：HTML・ベクターPDF・Word出力

長文仕様書・白書 (`type = "doc"`)

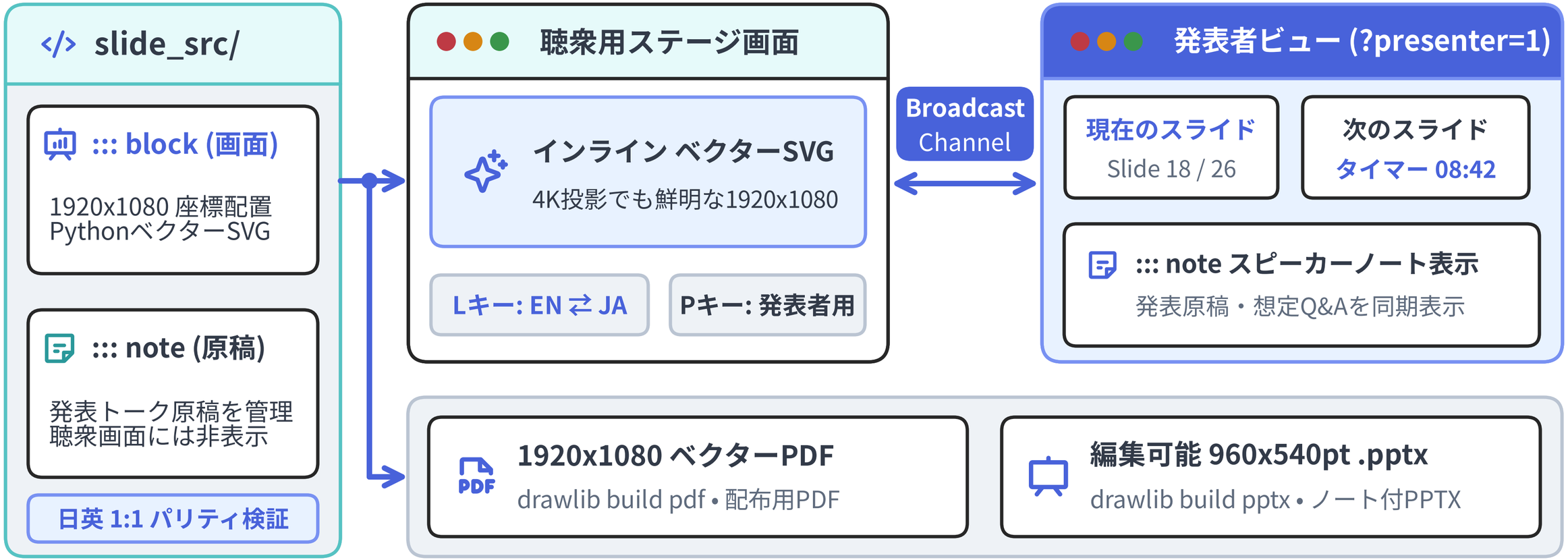
- **章ファイルの自動結合**: `00_cover.md`, `01_intro.md` を単一文書へ統合
- **表紙・目次の自動生成**: `--generate-index` で表紙とリンク目次を構築
- **3形式へ同時出力**: HTML・A4ベクターPDF・編集可能な `.docx` を生成

```
# doc_src/ から3形式を一括ビルド
uv run drawlib build html
uv run drawlib build pdf --generate-index
uv run drawlib build docx --generate-index
```



16:9 スライド・発表者ビュー・PPTX出力

- **2層構造のスライド記述**: `::: block` で `1920×1080` の視覚ステージを定義し、 `::: note` に発表原稿をGit管理。
- **リアルタイム同期とOffice出力**: `?presenter=1` で2画面発表者ビューを `BroadcastChannel` 同期し、PDFと `.pptx` も出力。



CHAPTER

06

Chapter 6 / 6

AIエージェント自律ワークフロー

Rules CLI・視覚的自己修正ループ・SQLiteキャッシュ

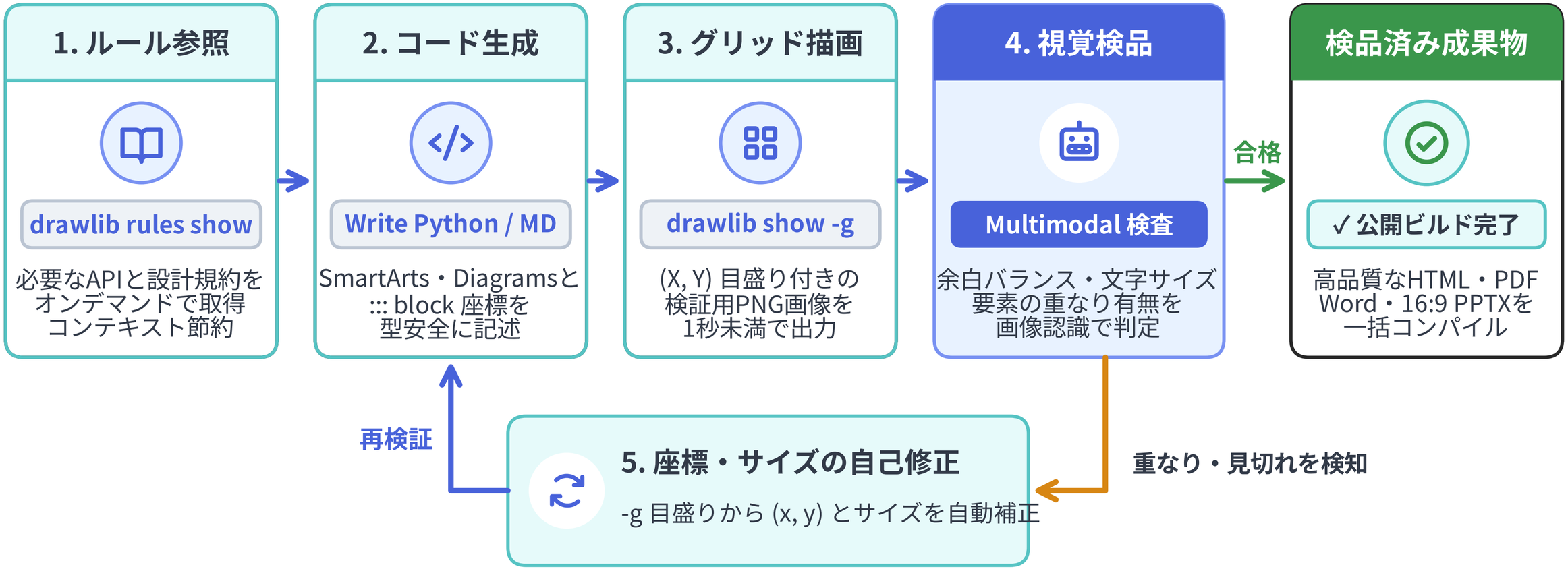
6.1 Rules CLIによるオンデマンド参照

6.2 -g グリッドによる自律視覚検証ループ

6.3 2層アセット＆SQLiteビルドキャッシュ

マルチモーダル視覚検証による自律修正ループ

- **オンデマンド仕様参照**: AIエージェント（Gemini・Claude Code・Cursor）が `drawlib rules show` で必要な知識のみを都度取得。
- **視覚的な自己修正ループ**: `drawlib show -g` の座標グリッド画像をマルチモーダル検品し、重なりや余白を自律修正して出力。



2層キャッシュ機構と統合CLIコマンド

< 1s

高速差分ビルド

SQLite SHA-256 .drawlib/cache.db

50+

AIルールトピック

drawlib rules show で都度参照

Zero

リンク切れゼロ

drawlib serve --check で自動検証

2層キャッシュアーキテクチャ

Tier 1: アセットキャッシュ

drawlib cache list / download

フォント・34種アイコン・Chromiumを
オンデマンド取得し事前キャッシュ

Tier 2: ビルドキャッシュ

.drawlib/cache.db (SQLite)

SHA-256内容＋スタイルハッシュ管理
HTML・PDF・Office間で高速共有

統合CLIワークフロー — uv run drawlib <command>



CLOSING 06 • まとめ

まとめと次のステップ

Illustration & Documentation as Code を始めよう

まとめとクイックスタート

01

1. Pure Python & Git ネイティブ

型安全なPython API・styles.py 共通設定と
Git差分管理でGUIロックインを解消。

02

2. 豊富な高水準コンポーネント群

18種SmartArts・9種Charts・6種Diagrams・
5種Graph・34種アイコンを標準内蔵。

03

3. 5つのプロジェクト種別とOffice出力

Site・Catalog・Doc・Slide・Images から
HTML・PDF・Word・PPTXを一括生成。

04

4. 自律型AIエージェントに最適化

50超の drawlib rules・show -g 視覚検品・
1秒未満の .drawlib/cache.db 差分ビルド。

30秒で始めるクイックスタート

Python 3.12+ / uv 環境に対応

```
uv add drawlib
```

→

```
uv run drawlib init project slide my-deck
```

→

```
uv run drawlib serve
```

ご清聴ありがとうございました

図解もドキュメントも、すべてコードで。

- **Docs**: `https://www.drawlib.com/`
- **API Catalog**: `https://www.drawlib.com/api/`
- **Examples & GitHub**: `https://examples.drawlib.com/` ・ `github.com/yuichi110/drawlib`

